

VENDLY.LK IMPLEMENTATION MANUAL

Seller Business Assistant Architecture & Step-by-Step Build Guide

A source-aligned guide to permission-aware dashboard search, navigation, analytics, exports, confirmed writes, audit logging and voice interaction.

Audience: a learner rebuilding the seller assistant from scratch with React, Flask and Firestore

CURRENT PROJECT IMPLEMENTATION • AUGUST 2026

Workspace: D:\Documents\orderflow\vendly-lk-web

How to use this guide

Build the feature in order. Each section explains the reason first, shows a small code fragment, and then gives a verification step. Paths are relative to the Vendly project root unless an absolute path is shown.

Part	Purpose
1	Architecture and safety model
2	Files, authentication and permissions
3	Assistant endpoint and response contract
4	Deterministic intents and AI fallback
5	Read-only business operations
6	Client actions and dashboard navigation
7	Confirmed write operations and audit logs
8	React panel, cards and voice
9	Testing, extension and production readiness

Learning rule

Type the fragments yourself, run the verification after each stage, and commit only when that stage works. Do not paste the entire feature at once.

1. Architecture and safety model

The Business Assistant is not a general autonomous agent. It is a permission-aware command interpreter placed above existing Vendly services. It may summarize and navigate immediately, but any stock or order mutation is prepared first and executed only after explicit seller confirmation.

Safety-first request lifecycle

```

Seller dashboard (React BusinessAssistant)
  |
  | Firebase bearer token + businessId + message
  v
Flask endpoint /businesses/{businessId}/assistant/messages
  |
  |--> require_firebase_user
  |--> require_business_member
  |--> deterministic_intent(message)
  |    \--> optional AI classification fallback
  |
  |--> process_read_intent() -----> immediate response/clientAction
  \--> prepare_write_intent() ----> confirmation card
        |
        | seller confirms
        v
        confirm_action() --> domain service --> Firestore + audit log

```

Rule	Implementation
Tenant isolation	Every request includes businessId and verifies an active membership document.
Least privilege	Each intent calls require_permission() for orders, products, customers or analytics.
No direct AI writes	AI returns a structured intent; server resolves real IDs and allowed values.
Human confirmation	Mutations return pendingAction and require a second request with confirmedAction.
Allowlist	Only update_order_status and adjust_stock are executable confirmed actions.
Auditability	Confirmed actions are copied into assistantAuditLogs with actor UID and result IDs.

2. Files, authentication and permissions

Current file	Responsibility
frontend/src/components/BusinessAssistant.jsx	Floating panel, messages, suggestions, cards, confirmations, exports and voice.
frontend/src/components/BusinessAssistant.css	Panel, mobile layout, message and action styling.
frontend/src/services/businessAssistantService.js	POST wrapper for message or confirmedAction.
backend/app/api/business_assistant.py	Authenticated, rate-limited Flask endpoint.

Current file	Responsibility
backend/app/services/business_assistant_service.py	Intent classification, permission checks, reads, prepared writes and audits.
backend/app/services/ai_service.py	Optional structured intent classification.
backend/app/core/auth.py	Firebase ID-token verification and verified-email rule.
backend/app/core/authorization.py	Membership and resource permission enforcement.

2.1 Protect the endpoint

backend/app/api/business_assistant.py

```
@business_assistant_blueprint.post(
    "/businesses/<business_id>/assistant/messages"
)
@limiter.limit("30 per minute")
@require_firebase_user
@require_business_member()
def business_assistant_message(business_id):
    response = handle_business_assistant_message(
        get_firestore_client(), business_id,
        g.current_user["uid"], g.membership, get_json_object(),
    )
    return jsonify({"assistant": response})
```

Decorator order matters: requests are rate-limited, authenticated and authorized before the service receives a database client or membership.

2.2 Verify fine-grained permission inside each intent

```
def require_permission(membership, permission):
    if not membership_has_permission(membership, permission):
        raise ApiError(
            "permission_denied",
            "Your staff role does not allow this assistant action.",
            403,
            {"requiredPermission": permission},
        )
```

Intent family	Suggested permission
Order search/filter/status	orders:read / orders:update
Inventory search/adjust	products:read / products:update
Customer search/export	customers:read / customers:export
Business summary	analytics:read
Settings/staff	settings:read or members:manage

3. Assistant endpoint and response contract

Stable JSON contract

```
// Request: natural-language message
{ "message": "Show packed orders" }

// Response
{
  "assistant": {
    "message": "Opening packed orders.",
    "cards": [],
    "suggestions": ["Reset filters"],
    "navigation": "/orders?status=packed",
    "clientAction": null,
    "pendingAction": null
  }
}

// Request: confirm a prepared mutation
{ "confirmedAction": { "type": "adjust_stock", ... } }
```

Field	Meaning
-------	---------

Field	Meaning
message	Text shown in the assistant bubble.
cards	Small order/product/business summary objects for visual rendering.
suggestions	Safe next prompts displayed as buttons.
navigation	Internal React route the browser may open.
clientAction	Browser-only action such as export, modal open, settings panel or reset filters.
pendingAction	Server-prepared mutation requiring explicit confirmation.

frontend/src/services/businessAssistantService.js

```
export async function sendBusinessAssistantMessage(
  businessId,
  { message = "", confirmedAction = null } = {},
) {
  const response = await apiRequest(
    `/businesses/${businessId}/assistant/messages`,
    { method: "POST", body: confirmedAction ? { confirmedAction } : { message } },
  );
  return response.assistant;
}
```

4. Deterministic intents and AI fallback

Fast, common commands are classified with Python first. Only unknown language is sent to the optional provider. This keeps the assistant fast, inexpensive and testable.

```
def classify_intent(message):
    deterministic = deterministic_intent(message)
    if deterministic["intent"] != "unknown":
        return deterministic
    return generate_business_assistant_intent(message) or deterministic
```

4.1 Build exact deterministic patterns

```
def deterministic_intent(message):
    lowered = message.casefold().strip()
    if lowered in {"help", "what can you do"}:
        return {"intent": "help"}
    if "export orders" in lowered:
        return {"intent": "export_orders"}
    if "add product" in lowered:
        return {"intent": "open_add_product"}
    if "low stock" in lowered:
        return {"intent": "low_stock"}
    match = ORDER_NUMBER_PATTERN.search(message)
    if match:
        return {"intent": "search_order", "orderQuery": match.group(0)}
    return {"intent": "unknown"}
```

4.2 Force AI to return structured JSON

```
ALLOWED_INTENTS = {
    "business_summary", "search_order", "search_product", "order_view",
    "inventory_view", "update_order_status", "adjust_stock", "navigate"
}

prompt = (
    "Return one JSON object only. Use an allowed intent. "
    "Never invent an ID, status or quantity not present in SELLER MESSAGE. "
    "For a stock change return productQuery, optional variantQuery and signed "
    "quantityChange. For an order update return orderQuery and status."
)
```

Validate after AI

AI output is untrusted. Recheck intent, status, quantity, record match and membership before acting.

5. Read-only business operations

Intent	Server behavior	Typical output
business_summary	get_business_analytics()	Revenue, order and stock summary cards.
pending_orders	list_orders(status='needs-confirmation')	Count and order cards.
low_stock	list_products() then filter stockStatus	Product cards and inventory link.
search_order	Find exact order number or matching customer.	One or more order cards.
search_product	Token-match product name/SKU/barcode.	Product cards.
order_view	Build /orders query parameters.	Internal navigation.
inventory_view	Build /inventory query parameters and sort.	Internal navigation.
customer_view	Open customer page with query.	Internal navigation.

```
if intent_name == "business_summary":
    require_permission(membership, "analytics:read")
    analytics = get_business_analytics(database, business_id)
    return {
        "message": "Here is today's business summary.",
        "cards": [{
            "type": "summary",
            "orders": analytics["orders"]["all"],
            "revenue": format_money(analytics["revenueMinor"]),
            "lowStock": analytics["inventory"]["lowStock"],
        }],
    }
```

Reuse existing services instead of duplicating Firestore queries inside the assistant. The assistant becomes an orchestration layer, while analytics/order/product services remain the single source of truth.

6. Client actions and dashboard navigation

clientAction.type	React behavior
export_orders	Call downloadOrderExport() and save the returned file.
export_inventory	Load products and call downloadInventoryCsv().
export_customers	Load customers and call downloadCustomersCsv().
open_settings	Dispatch a settings event with the requested section.
reset_filters	Dispatch vendly:reset-filters for the active page.
open_add_order/product/courier	Navigate and dispatch an event that opens the corresponding modal.

frontend/src/components/BusinessAssistant.jsx

```
async function executeResponseAction(response) {
    const action = response?.clientAction;
    if (action?.type === "export_orders") {
        await downloadOrderExport(business.id);
    }
    if (action?.type === "open_settings") {
        window.dispatchEvent(new CustomEvent("vendly:open-settings", {
            detail: { section: action.section || "general" },
        }));
    }
    if (response?.navigation) navigate(response.navigation);
}
```

Keep UI work in the browser

The backend describes the action; React owns navigation, modal visibility and file downloads. This avoids coupling Flask to browser implementation details.

7. Confirmed write operations and audit logs

7.1 Prepare, do not execute

```
if intent_name == "update_order_status":
    require_permission(membership, "orders:update")
    orders = find_order(database, business_id, intent.get("orderQuery"))
    if len(orders) != 1:
        return {"message": "I need one exact order number."}
    order = orders[0]
    return {
        "message": f"Please confirm: change {order['orderNumber']} to {status}?",
        "pendingAction": {
            "type": "update_order_status",
            "orderId": order["id"],
            "status": status,
            "label": f"Mark {order['orderNumber']} as {status}",
        },
    }
```

7.2 Confirm through an allowlist

```
ALLOWED_CONFIRMED_ACTIONS = {"update_order_status", "adjust_stock"}

def confirm_action(database, business_id, uid, membership, action):
    if action.get("type") not in ALLOWED_CONFIRMED_ACTIONS:
        raise ApiError("invalid_assistant_action", "Action is not allowed.", 422)
    if action["type"] == "update_order_status":
        require_permission(membership, "orders:update")
        order = update_order_status(
            database, business_id, action["orderId"], uid,
            {"status": action["status"],
             "note": "Updated through the Vendly business assistant"},
        )
        audit_action(database, business_id, uid, action, order)
    return {"message": f"Done – {order['orderNumber']} is now {action['status']}."}
```

7.3 Save an audit event

```
def audit_action(database, business_id, uid, action, result):
    database.collection("businesses").document(business_id)
    .collection("assistantAuditLogs").document().set({
        "actorUid": uid,
        "action": action,
        "resultId": result.get("id"),
        "createdAt": firestore.SERVER_TIMESTAMP,
    })
```

Concurrency

Stock changes must use the existing stock service/transaction. A confirmation card is not a stock lock; revalidate available/reserved stock when the confirmed request executes.

8. React panel, cards and voice

8.1 Message flow

```

async function sendMessage(text) {
  const cleanMessage = text.trim();
  if (!cleanMessage || !business?.id || isSending) return;
  setMessages((items) => [...items, {
    id: messageId("seller"), role: "user", text: cleanMessage,
  }]);
  setIsSending(true);
  try {
    const response = await sendBusinessAssistantMessage(business.id, {
      message: cleanMessage,
    });
    appendAssistantResponse(response);
    await executeResponseAction(response);
  } finally {
    setIsSending(false);
  }
}

```

8.2 Confirm or cancel a prepared action

```

async function confirmAction(action, sourceMessageId) {
  markActionState(sourceMessageId, "confirmed");
  const response = await sendBusinessAssistantMessage(business.id, {
    confirmedAction: action,
  });
  appendAssistantResponse(response);
}

function cancelAction(sourceMessageId) {
  markActionState(sourceMessageId, "cancelled");
}

```

8.3 Voice input and spoken replies

```

const recognition = new SpeechRecognition();
recognition.lang = voiceLanguage; // en-LK or si-LK
recognition.interimResults = false;
recognition.onresult = (event) => {
  const transcript = event.results?.[0]?.[0]?.transcript?.trim() || "";
  if (transcript) sendMessage(transcript);
};

function speak(text) {
  if (!speechEnabled || !window.speechSynthesis || !text) return;
  const utterance = new SpeechSynthesisUtterance(text);
  utterance.lang = voiceLanguage;
  window.speechSynthesis.cancel();
  window.speechSynthesis.speak(utterance);
}

```

- Always provide text input; voice is optional progressive enhancement.
- Show listening state and a stop control.
- Cancel speech synthesis when the panel closes or a new reply starts.
- Never execute a mutation directly from a transcript; confirmation still applies.

9. Testing, extension and production readiness

9.1 Minimum tests

Test	Expected result
Unauthenticated request	401 before any assistant work.
User outside business	403 tenant access denied.
Staff missing permission	403 with requiredPermission detail.
Deterministic export/navigation	Correct clientAction without AI call.
AI returns invalid JSON	Falls back to help/unknown safely.
Ambiguous order/product	Asks for one exact match; no write.
Write request	Returns pendingAction; database unchanged.
Tampered confirmedAction	Rejected by type/status/quantity validation.
Valid confirmation	Shared service changes data and audit log is written.
Concurrent stock change	Domain transaction prevents invalid stock.

9.2 Add a new intent safely

1. Name the intent and decide whether it is read-only, browser-only or mutating.
2. Add deterministic keywords for common phrasing.
3. If AI may classify it, add the name and exact JSON fields to the prompt allowlist.
4. Add permission enforcement before reading or preparing data.
5. For a browser action, return clientAction and implement it in React.
6. For a mutation, prepare a minimal pendingAction containing resolved server IDs.
7. Add the type to ALLOWED_CONFIRMED_ACTIONS only after confirm_action validates it.
8. Call an existing domain service and write an audit record.
9. Add unit tests for permission denied, ambiguous target, tampering and success.

9.3 Recommended next capabilities

Capability	Safe first version
Courier recommendation	Read-only ranked explanation; seller selects courier.
Dead-stock report	Read product age and sales history; navigate to filtered inventory.
Profit question	Read-only analytics cards using verified cost and sales data.
Daily work centre	Return actionable counts and links, not autonomous writes.
Create promotion	Prepare a draft campaign; require confirmation before publish.
Role management	Open settings only; do not let natural language change owners initially.

9.4 Production checklist

- Use HTTPS and exact CORS origins; never wildcard credentialed production requests.
- Use Redis-backed shared rate-limit storage when running multiple backend instances.
- Keep AI keys and Firebase Admin credentials on Flask only.
- Record provider latency/errors without logging seller messages containing personal data.
- Apply timeouts and retain deterministic behavior when the AI provider fails.
- Monitor assistantAuditLogs and alert on repeated rejected/tampered actions.
- Expose a visible cancel/close button and preserve keyboard accessibility.

Completion definition

The Business Assistant is production-ready when every request is tenant-scoped, permission-checked, deterministic where possible, resilient without AI, confirmation-gated for writes, and auditable after execution.